FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
FIAP

Caio Eduardo Vieira Da Silva
RM 96020

Luis Felipe Fernandes de Oliveira
RM 96131

Thierry Nathan Pereira De Souza
RM 96186

Relatório Técnico de Testes de Segurança e Mitigações

Plataforma de contratações online

São Paulo – SP

2025

1. INTRODUÇÃO E CONTEXTO

O presente relatório técnico aborda a análise de vulnerabilidades e a proposição de um roteiro de testes de segurança da informação para uma plataforma de contratações online que lida com dados sensíveis (currículos e dados pessoais) e é alvo constante de ataques de Negação de Serviço (DoS) e invasões diretas ao banco de dados.
O objetivo é fornecer uma visão robusta das defesas necessárias, baseada nas melhores práticas de mercado e no mapeamento das vulnerabilidades críticas do OWASP Top 10.

2. ROTEIRO DE TESTES E ANÁLISE

2.1. Vulnerabilidades Mapeadas (OWASP Top 10)

A análise de vulnerabilidades deve seguir uma abordagem sistemática, priorizando os pontos fracos mais críticos conforme o contexto (dados sensíveis, ataques DoS e ataques a banco de dados). As **seis** principais vulnerabilidades identificadas, com base no **OWASP Top 10 (2021)**, são:

VULNERABILIDADE 1: A03:2021 - Injection (Injeção)

- **Análise:** Dado que a plataforma sofre ataques diretos ao banco de dados, é altamente provável que existam falhas de tratamento de entrada de dados, especialmente via *SQL Injection*. Isso pode ocorrer em campos de pesquisa, formulários de cadastro de currículos ou parâmetros de URL, permitindo que atacantes manipulem comandos SQL para extrair, modificar ou deletar informações.
- **Teste de Segurança:** Teste de Injeção de SQL (Ex: inserir ' OR '1'='1 em campos de login ou pesquisa) e Injeção de Comandos (OS Command Injection) em APIs que interagem com o sistema operacional.

VULNERABILIDADE 2: A01:2021 - Broken Access Control (Controle de Acesso Quebrado)

- **Análise:** Uma plataforma que gerencia currículos possui diferentes níveis de acesso (candidato, recrutador, administrador). Uma falha no controle de acesso pode permitir que um usuário de menor privilégio (ex: um candidato) acesse dados de outros candidatos, informações confidenciais de recrutadores ou até mesmo endpoints administrativos.
- **Teste de Segurança:** Teste de IDOR (Insecure Direct Object Reference) tentando acessar dados (ex: user/123 para user/124) e teste de escalonamento de privilégios.

VULNERABILIDADE 3: A05:2021 - Security Misconfiguration (Configuração Incorreta de Segurança)

- **Análise:** Ataques em servidores (como os que precedem os ataques DoS ou DB) frequentemente exploram configurações padrão e não otimizadas. Isso inclui o uso de

credenciais padrão, serviços de rede desnecessariamente expostos, diretórios ou logs acessíveis publicamente, ou falhas no *patch management* do sistema operacional e do servidor de aplicação (Tomcat, Apache, Nginx).
- **Teste de Segurança:** Escaneamento de portas e serviços abertas, verificação de cabeçalhos de segurança HTTP e testes de acesso a caminhos e diretórios sensíveis (/admin, .git, /logs).

VULNERABILIDADE 4: A07:2021 - Identification and Authentication Failures (Falhas de Identificação e Autenticação)

- **Análise:** O gerenciamento de milhares de currículos implica milhares de contas de usuário. Políticas de senha fracas, falta de Multi-Factor Authentication (MFA), ou falhas na gestão de sessões podem levar ao comprometimento de contas e subsequente roubo de dados, ou uso da conta para fins maliciosos.
- **Teste de Segurança:** Testes de força bruta e dicionário, verificação da robustez do processo de recuperação de senha e análise da validade e do ciclo de vida dos tokens de sessão.

VULNERABILIDADE 5: A02:2021 - Cryptographic Failures (Falhas Criptográficas)

- **Análise:** A plataforma lida com dados de identificação pessoal (PII) e informações confidenciais de currículos. A ausência de criptografia forte (ex: uso de algoritmos legados, armazenamento de senhas em *plaintext* ou criptografia insuficiente de dados em repouso) expõe os usuários em caso de violação.
- **Teste de Segurança:** Análise da implementação de HTTPS/TLS, verificação do algoritmo de hashing de senhas (deve ser bcrypt ou Argon2, nunca MD5 ou SHA-1) e inspeção das configurações de criptografia do banco de dados (TDE).

VULNERABILIDADE 6: A04:2021 - Insecure Design (Design Inseguro)

- **Análise:** O design da aplicação pode permitir que usuários mal-intencionados explorem a funcionalidade de upload de currículos. Por exemplo, aceitar uploads de arquivos executáveis (.exe) ou que contenham macros maliciosas sem validação, ou armazenar esses arquivos dentro do diretório web acessível publicamente.
- **Teste de Segurança:** Tentativa de upload de arquivos com extensões não permitidas (Ex: .php, .js, .bat), teste de limite de tamanho de arquivo e verificação da localização de armazenamento dos arquivos carregados.

2.2. Ações Preventivas e Mitigação

A prevenção deve ser baseada no princípio da Defesa em Profundidade (*Defense in Depth*), garantindo múltiplas camadas de proteção.

Vulnerabilidade	Ação Preventiva Proposta	Descrição da Ação
Injection (A03)	**Uso de Prepared Statements (Consultas Parametrizadas)**	Em vez de concatenar *strings* do usuário diretamente no código SQL, utilize Prepared Statements. O banco de dados

Vulnerabilidade	Ação Preventiva Proposta	Descrição da Ação
		trata o input do usuário estritamente como dados, e não como parte do comando, neutralizando a maioria dos ataques de injeção.
Broken Access Control (A01)	**Implementação de RBAC e Princípio do Menor Privilégio**	Adotar o Role-Based Access Control (RBAC) rigoroso, garantindo que o código verifique a função e as permissões do usuário em cada requisição que acessa recursos ou dados. Além disso, a aplicação e os usuários devem ter o menor nível de privilégio necessário para realizar suas tarefas (Least Privilege).
Security Misconfiguration (A05)	**Hardening do Servidor e Automação de Patches**	Implementar uma rotina de *hardening* (endurecimento) nos servidores de forma automatizada (usando ferramentas IaC - Infrastructure as Code, como Terraform ou Ansible) e manter uma política rigorosa de Patch Management para garantir que sistemas operacionais, bibliotecas e serviços estejam sempre na versão mais recente e segura.
Identification and Authentication Failures (A07)	**Obrigatoriedade do Multi-Factor Authentication (MFA)**	Tornar obrigatório o uso de Multi-Factor Authentication (MFA) para todos os usuários, especialmente para contas administrativas e de recrutadores. Isso adiciona uma camada de segurança que impede que senhas roubadas por si só resultem em acesso.
Cryptographic Failures (A02)	**Hashing de Senhas com Algoritmos Modernos e Criptografia Total**	Usar algoritmos de *hashing* lentos e salgados (como bcrypt ou Argon2) para todas as senhas. Além disso, garantir que dados sensíveis armazenados no banco de dados sejam criptografados em repouso (TDE) e que toda a

Vulnerabilidade	Ação Preventiva Proposta	Descrição da Ação
		comunicação utilize TLS/SSL robusto.
Insecure Design (A04)	**Validação Rigorosa de Arquivos e Isolamento de Storage**	Implementar uma abordagem de *whitelist* (lista de permissão) para extensões de arquivos de currículos (apenas .pdf, .docx, etc.). Armazenar os arquivos em um local isolado (*Storage Bucket*) fora da raiz da web do servidor de aplicação e aplicar *scanning* contra malware antes do processamento.

2.3. Prevenção de Ataques no Banco de Dados

Atuar no banco de dados exige um foco em segmentação, criptografia e monitoramento, tratando o banco de dados como o ativo mais crítico da plataforma.

1. **Segmentação de Rede (Network Segmentation):**
    - O banco de dados nunca deve ser acessível diretamente pela internet. Deve residir em uma sub-rede privada (VPC/VNet) e apenas o servidor de aplicação deve ter permissão de conexão (via portas e IPs específicos).
    - Utilizar *Database Firewall* ou políticas de segurança de rede para isolar o banco de dados do servidor de aplicação de forma lógica, limitando o tráfego a portas específicas (ex: 3306 para MySQL ou 5432 para PostgreSQL).
2. **Criptografia Completa (Encryption at Rest and in Transit):**
    - **Em Repouso (At Rest):** Utilizar a criptografia de disco transparente (TDE) fornecida pelo provedor de banco de dados ou pelo sistema operacional.
    - **Em Trânsito (In Transit):** Exigir o uso de SSL/TLS (Secure Socket Layer / Transport Layer Security) para todas as conexões entre o servidor de aplicação e o banco de dados, protegendo os dados durante a transferência.
3. **Princípio do Menor Privilégio para Usuários de Aplicação:**
    - O usuário que a aplicação web usa para se conectar ao banco de dados não deve ter privilégios de administrador. Deve ter apenas permissões de SELECT, INSERT, UPDATE e DELETE nas tabelas estritamente necessárias. **Nunca deve ter permissões para criar (CREATE), deletar (DROP) ou alterar o esquema do banco de dados (ALTER).**
4. **Monitoramento de Atividade (Database Activity Monitoring - DAM):**
    - Implementar uma ferramenta de DAM para monitorar e auditar todas as consultas ao banco de dados em tempo real. Isso permite detectar e alertar sobre consultas anômalas, como grandes volumes de extração de dados (data exfiltration) ou comandos administrativos executados em horários não usuais.

2.4. Barreiras de Proteção contra DoS/DDoS

A Negação de Serviço (DoS/DDoS) é um ataque volumétrico que visa sobrecarregar os recursos. A defesa deve ser distribuída na camada de rede (borda) e na camada de aplicação.

1. **Web Application Firewall (WAF) e Serviços de Mitigação de DDoS:**

    - **Implementação na Borda:** Utilizar um serviço de CDN (Content Delivery Network) ou um provedor de mitigação de DDoS (como Cloudflare, Akamai ou AWS Shield) que atue como a primeira linha de defesa. Estes serviços absorvem o tráfego volumétrico e filtram pacotes maliciosos antes que cheguem à infraestrutura da plataforma.
    - **WAF:** O WAF deve ser configurado para bloquear padrões conhecidos de ataque, impor limites de tamanho de requisição e defender contra ataques da camada de aplicação (L7), como ataques lentos (*Slowloris*) e ataques de sobrecarga de recursos HTTP.
2. **Limitação de Taxa (Rate Limiting) e Contramedidas de Bot:**
    - **Limitação por IP/Sessão:** Implementar regras de *Rate Limiting* (limitação de taxa) no balanceador de carga ou no WAF para restringir o número de requisições que um único endereço IP ou sessão pode fazer em um determinado período (ex: 100 requisições por segundo).
    - **Desafios para Bots:** Utilizar desafios como *CAPTCHA* ou verificações de JavaScript (como o Turnstile da Cloudflare) para distinguir entre tráfego humano e robótico excessivo, bloqueando os bots de ataque.
3. **Balanceamento de Carga e Auto-Scaling:**
    - **Load Balancers:** Utilizar *Load Balancers* distribuídos para dividir o tráfego de entrada por vários servidores de aplicação. Se um servidor estiver sobrecarregado, o *Load Balancer* redireciona o tráfego para servidores menos ocupados.
    - **Auto-Scaling (Escalabilidade Automática):** Configurar grupos de *Auto-Scaling* para que, em caso de aumento súbito e legítimo de tráfego, novos servidores de aplicação sejam automaticamente provisionados para absorver a carga, aumentando a resiliência do serviço.
4. **Otimização e Cache:**
    - Garantir que os recursos estáticos (CSS, imagens, JS) estejam em cache no CDN. Isso reduz a carga sobre os servidores de origem.
    - Otimizar a aplicação para que as requisições de maior consumo de recursos sejam as últimas a serem executadas e sejam protegidas por mecanismos de cache ou de processamento assíncrono.

3. CONCLUSÃO

A análise realizada demonstra que a plataforma de contratações online apresenta um conjunto de riscos compatíveis com sistemas que lidam com dados sensíveis, grande volume de requisições e armazenamento de currículos. As vulnerabilidades identificadas, mapeadas diretamente ao OWASP Top 10, evidenciam que os principais vetores de ataque envolvem falhas de controle de acesso, configurações inadequadas, riscos de injeção e fragilidades no design que podem ser exploradas tanto por agentes automatizados quanto por atacantes experientes.

As ações preventivas apresentadas adotam uma abordagem de **Defesa em Profundidade (Defense in Depth)**, garantindo múltiplas camadas de proteção, desde a aplicação até o banco de dados e a camada de borda da rede. Além disso, as recomendações incluem práticas amplamente reconhecidas no mercado, como MFA, criptografia robusta, segmentação de rede, RBAC, hardening automático e mitigação avançada contra DoS/DDoS.

É recomendável que a empresa incorpore essas práticas não apenas como medidas reativas, mas como parte contínua do **SDLC (Secure Development Lifecycle)**, integrando testes recorrentes de pentest, auditorias e processos de monitoramento ativo para manutenção da postura de segurança. Como a plataforma lida com um grande volume anual de dados pessoais, a adoção desse conjunto de medidas é essencial para garantir confidencialidade, integridade, disponibilidade e conformidade com padrões internacionais.

4. BIBLIOGRAFIA

OWASP Foundation. *OWASP Top 10 – 2021: The Ten Most Critical Web Application Security Risks.*
Disponível em: https://owasp.org/Top10/

Gupy. *Aviso de Privacidade para Candidatos – Medidas de Proteção e Tratamento de Dados.*

Disponível em: **https://www.gupy.io/en/aviso-de-privacidade-candidatos**

NIST – National Institute of Standards and Technology. *NIST SP 800-53 – Security and Privacy Controls for Information Systems and Organizations.*
Disponível em: https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final

NIST. *NIST SP 800-63 – Digital Identity Guidelines.*
Disponível em: https://pages.nist.gov/800-63-3/

Cloudflare. *DDoS Attack Prevention & Mitigation.*
Disponível em: https://www.cloudflare.com/learning/ddos/

Microsoft. *SQL Injection Overview & Prevention Guidelines.*
Disponível em: https://learn.microsoft.com/en-us/sql/relational-databases/security/sql-injection